

Doc 2: Technical Specifications

1. Monorepo Architecture (pnpm)

The project uses a strict `pnpm` workspace to share logic and types between the client and server without code duplication.

- > `apps/client/` : The Terminal UI application (rendering, inputs).
- > `apps/server/` : The Colyseus multiplayer game server.
- > `packages/shared/` : The "Source of Truth" (Colyseus state schemas, TypeScript interfaces, game constants). Both apps import from here.
- > `packages/map-engine/` : Map parsing logic and procedural generation utilities.

2. Industry-Proven Technology Stack

- > **Frontend Engine:** `terminal-kit`. The most battle-tested Node.js terminal library. Provides native screen buffers, 32-bit color, and raw input handling for a tear-free 60FPS experience.
- > **Multiplayer Backend:** `colyseus`. Authoritative TS multiplayer framework with built-in state synchronization (`@colyseus/schema`) and room instance routing.
- > **Database & ORM:** `prisma` with `SQLite` (for MVP) scaling to `PostgreSQL` (for production) to persist user accounts, wallets, and BBS posts.
- > **Game Logic Libraries:**
 - > `rot-js` (Roguelike Toolkit): Handles procedural dungeon generation (BSP, Cellular Automata) and Field of View calculations.
 - > `pathfinding` : Optimized A* (A-Star) routing for AI enemy movement.

3. AI Validation & Test-Driven Development (TDD)

The AI Agent must NOT guess if the code works. It must write automated tests to self-validate logic before marking any task as complete.

- > **Unit Testing (Vitest/Jest):** Write tests for all game math, map parsing, and utility functions in the `packages/` directory.
- > **State Testing:** Simulate client connections to the Colyseus server in test environments to verify that state updates and wallet deductions execute correctly.
- > **Constraint:** A feature is only considered "Done" if the corresponding test suite passes with 100% success.

4. Git Version Control Workflow

Strict version control is mandated. The AI must interact with the git CLI regularly.

- > Create a new feature branch for each Phase in the Roadmap (e.g., `feature/phase-1-netcode`).
- > Commit code **only** after the automated tests pass successfully.
- > Write clear, semantic commit messages (e.g., `feat(server): add colyseus room schema`).
- > Merge the feature branch into `main` at the conclusion of each Phase.